

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Algoritmos y Estructuras de Datos (CB100)

Trabajo Práctico 2: La fuga de la mazmorra

Grupo: JAVALOVERS

2º C 2026

Alumno	Padrón	Mail
Presas, Juan Cruz	115479	jxpresas@gmail.com
Martinez, Demian Arian	114977	deamartinez@fi.uba.ar
Solari Parravicini, Facundo	109324	fsolarip@fi.uba.ar
Caldas Canedo, Owen	115491	owenkurt105@gmail.com
Burgos, Matías Alejandro	115926	malej1.9@gmail.com
Ortega, Dante Luca	115569	danteluca06@gmail.com

HITO: 1

Fecha de entrega: 12/11/2026

Repositorio del proyecto: JAVALOVERS

Índice

1 Informe del trabajo realizado 4

1.1 Objetivo 4

1.2 Arquitectura y decisiones generales 4

1.2.1 Integración de la partida 4

1.3 Modelo del juego y reglas de la partida 4

1.3.1 Coordenadas y movimiento 4

1.3.2 Tablero y plano 5

1.3.3 Estado del héroe 5

1.3.4 Comportamiento común de las entidades 5

1.3.5 Organización de la mazmorra 5

1.3.6 Resolución del turno y puntaje 5

1.4 Generación de laberintos 6

1.4.1 Responsabilidad del módulo 6

1.4.2 Excavación y conectividad 6

1.4.3 Bifurcaciones y proporción transitable 6

1.4.4 Capacidad y reintentos 6

1.4.5 Determinismo 6

1.4.6 Entrada y salida 7

1.4.7 Entidades y terrenos 7

1.5 Validación y carga de planos 7

1.5.1 Responsabilidades del validador y del cargador 7

1.5.2 BFS compartido para conectividad y salida 8

1.5.3 Comprobación de porcentajes 8

1.5.4 Construcción del plano 8

1.6 Monstruos, cofres y efectos 8

1.6.1 Comportamiento de los monstruos 8

1.6.2 Apertura de cofres y puntaje 9

1.6.3 Creación de entidades mediante catálogos 9

1.7 Persistencia, configuración y vista 10

1.7.1 Lectura de configuración 10

1.7.2 Consola y representación del plano 10

1.8 Estructuras de datos 10

1.8.1 Pila de excavación del generador 10

1.8.2 Estructuras utilizadas por el validador 11

1.8.3 Estructuras del modelo y de la integración 11

1.8.4 Estructuras de los catálogos y del errante 11

1.9 Pruebas, integración e intercambio . . . 12
1.9.1 Pruebas del generador . . . 12
1.9.2 Pruebas del validador . . . 12
1.9.3 Integración del generador con el validador . . . 12
1.9.4 Pruebas del modelo y de la integración de la partida . . . 12
1.9.5 Pruebas del héroe, las entidades y los catálogos . . . 14
1.9.6 Pruebas de partida, configuración y vista . . . 14
1.9.7 Intercambio con otro grupo . . . 14
1.10 Problemas encontrados . . . 15
1.10.1 Generación de planos . . . 15
1.10.2 Validación y carga de planos . . . 15
1.10.3 Modelo e integración de la partida . . . 16
1.10.4 Implementación del héroe, las entidades y los catálogos . . . 16
1.11 Cuestionario . . . 17
1.12 Limitaciones . . . 18
1.12.1 Generación de planos . . . 18
1.12.2 Alcance de la integración en el Hito 1 . . . 18
1.12.3 Alcance de las entidades en el Hito 1 . . . 18
1.13 Aportes individuales . . . 18
1.13.1 Presas, Juan Cruz . . . 18
1.13.2 Martinez, Demian Arian . . . 19
1.13.3 Solari Parravicini, Facundo . . . 19
1.13.4 Caldas Canedo, Owen . . . 19
1.13.5 Burgos, Matías Alejandro . . . 19
1.13.6 Ortega, Dante Luca . . . 20
1.14 Conclusiones . . . 20
1.14.1 Resultados del Hito 1 . . . 20

2 Manual de usuario . . . 21
2.1 Descripción del programa . . . 21
2.2 Requisitos, instalación y ejecución . . . 21
2.3 Inicio de partida y selección de dificultad . . . 21
2.4 Pantallas, controles y reglas básicas . . . 21
2.5 Tareas frecuentes . . . 21
2.6 Archivos y configuración . . . 22
2.7 Mensajes de error y soluciones . . . 22

3 Manual del programador . . . 23
3.1 Preparación del entorno . . . 23

3.1.1 Compilación y ejecución con Gradle . . . 23
3.2 Estructura del proyecto . . . 23
3.3 Arquitectura y recorrido de una acción . . . 23
3.3.1 Recorrido de un movimiento . . . 23
3.4 Modelo y ejecución de la partida . . . 24
3.4.1 Uso de posiciones y entidades . . . 24
3.4.2 Contrato de Entidad . . . 24
3.4.3 Acceso a planos y resultado de una acción . . . 24
3.5 Generador de laberintos . . . 25
3.5.1 Ubicación y dependencias . . . 25
3.5.2 Contrato e invocación . . . 25
3.5.3 Modificación de los terrenos . . . 26
3.5.4 Sustitución del generador . . . 26
3.5.5 Consideraciones para modificar el código . . . 26
3.5.6 Selección de entrada y reintentos . . . 26
3.5.7 Errores . . . 26
3.6 Validador y cargador de planos . . . 27
3.6.1 Ubicación y responsabilidades . . . 27
3.6.2 Contrato del validador . . . 27
3.6.3 Contrato del cargador . . . 28
3.6.4 Interpretación del JSON . . . 28
3.6.5 Modificación de las comprobaciones . . . 28
3.6.6 Consideraciones de mantenimiento . . . 29
3.7 Monstruos, cofres y catálogos . . . 29
3.7.1 Agregar un tipo de monstruo . . . 29
3.7.2 Agregar un tipo de cofre . . . 29
3.7.3 Contrato de los catálogos . . . 29
3.7.4 Pruebas de un tipo nuevo . . . 30
3.8 Persistencia, configuración y vistas . . . 30
3.8.1 Lectura y validación . . . 30
3.8.2 Entrada y dibujo . . . 30
3.9 Ejecución y organización de pruebas . . . 30
3.9.1 Pruebas del generador . . . 30
3.9.2 Pruebas del modelo y del controlador . . . 31
3.10 Pendientes y problemas conocidos . . . 31
3.10.1 Estado al cierre del Hito 1 . . . 31

1. Informe del trabajo realizado

1.1. Objetivo

El objetivo del trabajo práctico es desarrollar en Java el juego *La fuga de la mazmorra*, aplicando los algoritmos y las estructuras de datos estudiados en la materia. El jugador debe guiar al héroe hasta la salida administrando su energía, que disminuye según el terreno recorrido y los encuentros con monstruos. Durante la partida también puede abrir cofres para obtener puntos.

Como primer hito, se buscó completar una partida en dificultad Fácil de principio a fin. Para ello se integraron la lectura de la configuración, la generación y validación del laberinto, el movimiento por consola y la interacción con monstruos errantes y cofres de puntos. El estado inicial y el resultado de cada paso válido se representan mediante imágenes BMP.

1.2. Arquitectura y decisiones generales

1.2.1. Integración de la partida

La clase `Main` lee la configuración, solicita la dificultad y conecta la partida con la vista y el controlador. El menú y la lectura de acciones utilizan una misma instancia de `EntradaConsola`.

`CreadorPartida` recibe la configuración y una implementación de `GeneradorDeLaberinto`. Según la dificultad elegida, distribuye los cofres y monstruos entre los planos y solicita una salida para el último. Cada JSON generado pasa por `CargadorDePlano`. Con los planos cargados se construye la mazmorra y se ubica al héroe en la entrada del primer plano.

La semilla se toma de la configuración. Si su valor es `null`, se sortea una al iniciar la partida. Para generar cada plano, el creador envía por JSON la semilla de la partida más el índice del plano, comenzando en cero. El generador crea su propio `Random` con ese valor. Los catálogos y los errantes comparten otra instancia inicializada con la semilla de la partida.

`ControladorJuego` muestra el estado inicial y solicita acciones mediante la interfaz `Vista`. Después de cada paso válido muestra el estado actualizado. Si el héroe intenta atravesar una pared, el controlador avisa al jugador y conserva el número de paso. El ciclo termina al ganar, perder o elegir salir.

1.3. Modelo del juego y reglas de la partida

1.3.1. Coordenadas y movimiento

`Posicion` representa una fila y una columna. `Ubicacion` agrega el número de tablero y de piso para identificar una celda dentro de la mazmorra. Las coordenadas públicas del modelo comienzan en uno, igual que en el JSON de intercambio.

Ambas clases se implementaron como `record`. Esto permite comparar dos posiciones por sus coordenadas y utilizarlas como claves en una colección. Cada valor de `Direccion` define su desplazamiento de fila y columna. El método `Posicion.mover` aplica ese desplazamiento y devuelve una nueva posición. Así se pueden conservar y comparar el origen y el destino de un movimiento.

1.3.2. Tablero y plano

`Tablero` almacena el terreno de cada celda. Un tablero nuevo comienza con todas sus celdas en pared y el cargador asigna los terrenos mediante `setTerreno`. El tablero permite consultar si una posición es transitable y obtener sus vecinas ortogonales transitables.

`Plano` reúne el tablero, las entidades, la entrada y la salida. En los planos sin salida, esta última se representa con `null`. La separación entre ambas clases permite probar los límites y movimientos sobre la grilla sin tener que construir cofres ni monstruos.

1.3.3. Estado del héroe

La clase `Heroe` conserva la ubicación, la energía y la cantidad de cofres abiertos. La partida consulta al tablero si el movimiento es válido y determina el costo del terreno. Una vez resuelta la acción, se actualiza el estado del héroe.

Al descontar energía, el valor mínimo es cero. Por ejemplo, si el héroe tiene 20 unidades y un errante le quita 30, la energía queda en cero y la partida evalúa su estado. Una cantidad negativa como argumento de `restarEnergia` se rechaza mediante una excepción.

1.3.4. Comportamiento común de las entidades

`Entidad` define cómo actúa una entidad durante un turno, cómo responde al encontrarse con el héroe y qué símbolo la representa. También permite consultar si debe retirarse del plano mediante `debeSalirDelPlano()`.

Por ejemplo, al abrirse un cofre, este indica que debe salir del plano y la partida lo retira. Así, `Partida` puede resolver el encuentro mediante el contrato común de las entidades. La vista utiliza el símbolo de cada entidad para elegir su representación.

1.3.5. Organización de la mazmorra

`Mazmorra` conserva los planos en una lista ordenada por tablero y, dentro de cada tablero, por piso. La entrada se obtiene del primer plano, en el tablero 1 y piso 1. Para determinar la ubicación de la salida se busca el plano que la contiene y se recuperan sus números de tablero y piso.

1.3.6. Resolución del turno y puntaje

`Partida` comprueba si el destino es transitable antes de cobrar energía. En la configuración del Hito 1, ingresar a piso cuesta 1 unidad, al agua 3 y a la lava 8. Un movimiento válido incrementa los pasos y resuelve el encuentro con las entidades de la celda. Esperar también cuenta como paso y conserva la energía del héroe por movimiento. En ambos casos, si la partida sigue en curso, se ejecuta el turno de los monstruos del plano actual.

Se verifica primero si el héroe alcanzó la salida. Por lo tanto, un último movimiento que consume toda la energía permite ganar si termina en esa celda. En otra ubicación, quedar sin energía produce la derrota. El puntaje se calcula como $100c + 2e - p$, donde c es la cantidad de cofres abiertos, e la energía restante y p el número de pasos. Tanto los puntos por cofre abierto y los puntos por energía son parámetros dentro de la clase `Partida`.

1.4. Generación de laberintos

1.4.1. Responsabilidad del módulo

El generador recibe los parámetros de un plano en formato JSON y devuelve su representación según el Anexo B. Implementa la interfaz `GeneradorDeLaberinto` sin depender de las clases del modelo del juego.

La grilla se representa mediante una matriz `char[][]`. Esta representación permite construir el plano sin crear objetos del dominio, cuya instanciación corresponde al cargador. Los records auxiliares se encuentran dentro de la implementación para permitir su intercambio en un único archivo.

1.4.2. Excavación y conectividad

Se utiliza backtracking iterativo con una pila `ArrayDeque`. Inicialmente, todas las celdas son paredes. Desde una posición interior se seleccionan destinos sin excavar a distancia dos y se abre también la celda intermedia.

La pila conserva el recorrido activo y permite retroceder cuando no existen destinos disponibles. Se evita así utilizar llamadas recursivas para almacenar el estado del recorrido. La conectividad se mantiene porque cada tramo nuevo se une a un camino existente.

1.4.3. Bifurcaciones y proporción transitable

Después de excavar se verifica que al menos el 10 % de las celdas transitables tenga tres o más vecinos transitables. Cuando no se alcanza ese porcentaje, se abren paredes interiores que conectan caminos existentes.

Las aperturas no deben superar el 60 % transitable. También se comprueba el mínimo del 30 %. Si la geometría no cumple estas condiciones, se realiza otro intento, hasta un máximo de cien. Este límite acota la ejecución, pero no garantiza encontrar una solución para toda entrada válida.

1.4.4. Capacidad y reintentos

Antes de generar se comprueba que la cantidad solicitada de entidades no supere una cota de capacidad. Esta considera las celdas interiores, el máximo del 60 % transitable y las posiciones reservadas para la entrada y la salida.

Durante cada intento se verifica que la grilla obtenida tenga espacio suficiente para todas las entidades y que exista una entrada compatible con la restricción de los monstruos.

Si alguna de estas condiciones falla, se construye una nueva grilla. El proceso admite hasta cien intentos, manteniendo la misma secuencia pseudoaleatoria. Así, una distribución particular insuficiente no provoca el rechazo inmediato de una solicitud que podría resolverse con otra grilla.

1.4.5. Determinismo

Se crea una instancia de `Random` por generación, inicializada con la semilla recibida. Todas las decisiones aleatorias utilizan esa misma instancia y las posiciones candidatas se recorren en un orden fijo.

Los reintentos continúan la secuencia pseudoaleatoria. Reiniciarla con la misma semilla repetiría el intento anterior. Con los mismos parámetros y la misma implementación se obtiene el mismo plano.

1.4.6. Entrada y salida

La entrada se selecciona al azar entre los pisos interiores, comprobando que permita ubicar todos los monstruos a distancia Manhattan de al menos tres. Cuando se solicita salida, también se reserva la posición que ocuparía para excluirla de las ubicaciones disponibles para monstruos.

Si una posición no cumple estas condiciones, se prueba otra sin repetir candidatos. Si ninguna resulta adecuada, se descarta la grilla y se realiza un nuevo intento.

La salida se obtiene mediante BFS desde la entrada, seleccionando una celda cuya distancia mínima sea máxima. Se utiliza la cantidad de movimientos, sin considerar el costo energético de los terrenos. En caso de empate se conserva la primera celda encontrada.

La condición de distancia Manhattan se concentra en el método `admiteMonstruo`, utilizado tanto al seleccionar la entrada como al colocar los monstruos.

1.4.7. Entidades y terrenos

Los monstruos se colocan antes que los cofres porque requieren una distancia Manhattan mínima de tres respecto de la entrada. Las posiciones elegidas se retiran de las listas de candidatos para evitar superposiciones.

El agua y la lava se colocan al final, únicamente sobre pisos libres. Los porcentajes elegidos se presentan en la Tabla 1.

Dificultad	**Agua**	**Lava**
Fácil	5 %	0 %
Medio	10 %	5 %
Difícil	10 %	10 %

Tabla 1: Porcentajes sobre el total de celdas transitables.

Las cantidades se redondean hacia abajo y se reducen cuando no hay suficientes pisos libres. Estos porcentajes son decisiones del grupo y respetan el máximo combinado del 25 %.

El generador determina las posiciones de las entidades; sus tipos concretos se seleccionan durante la carga del plano.

1.5. Validación y carga de planos

1.5.1. Responsabilidades del validador y del cargador

La clase `ValidadorPlano` recibe el JSON de un plano y comprueba las once reglas del Anexo B. Si encuentra una infracción, interrumpe la validación y lanza una `IllegalArgumentException`. El mensaje identifica la regla incumplida mediante un prefijo, como `R5:`.

Se eligió este mecanismo porque permite informar la causa del rechazo y comprobarla en los tests.

Un resultado booleano no permitiría distinguir las diferentes infracciones.

La construcción de los objetos del modelo corresponde a `CargadorDePlano`. Esta clase invoca al validador antes de crear el tablero y las entidades. La separación permite validar planos propios y de otros grupos sin depender de los catálogos del juego.

El JSON se interpreta en ambas clases. Se aceptó esta duplicación de lectura para mantener separadas la validación del formato y las reglas, y la construcción del modelo.

1.5.2. BFS compartido para conectividad y salida

Se realiza un recorrido en anchura desde la entrada para calcular las distancias mínimas en cantidad de movimientos. La matriz obtenida se utiliza tanto para comprobar la conectividad (R5) como para verificar que la salida se encuentre a distancia máxima de la entrada (R9).

BFS permite resolver ambas comprobaciones con un único recorrido. Un recorrido en profundidad permitiría verificar la conectividad, pero no obtendría directamente las distancias mínimas necesarias para R9.

Las paredes y las celdas no alcanzadas conservan el valor -1 en la matriz de distancias. Por eso, la conectividad se comprueba comparando la cantidad de celdas alcanzadas con el total de celdas transitables.

1.5.3. Comprobación de porcentajes

Las proporciones de bifurcaciones, celdas transitables y terrenos especiales se verifican mediante productos enteros, evitando divisiones que pierdan precisión.

Por ejemplo, la condición mínima de bifurcaciones se expresa comparando `bifurcaciones * 100` con `transitables * 10`. Esto permite incluir correctamente los casos que coinciden con el límite exigido.

1.5.4. Construcción del plano

El cargador construye un `Tablero` y registra las posiciones de entrada y salida. Los símbolos de pared, agua y lava se convierten en los terrenos correspondientes. Los símbolos de entrada, salida, cofre y monstruo se representan como piso en el tablero.

Por cada cofre o monstruo se solicita al catálogo correspondiente la creación de una entidad en esa posición. Los catálogos y la instancia de `Random` se reciben por constructor, de modo que el cargador no depende de tipos concretos de entidades.

El modelo utiliza coordenadas desde uno. Al recorrer las cadenas del JSON, cuyos índices comienzan en cero, el cargador suma uno a la fila y a la columna.

1.6. Monstruos, cofres y efectos

1.6.1. Comportamiento de los monstruos

Se utilizó el patrón *Template Method* para definir el turno común de los monstruos. El método `actuar` de `Monstruo` valida los datos recibidos, solicita al subtipo la próxima posición mediante

`decidirMovimiento` y actualiza la posición. Al producirse un encuentro, `alEncontrarseConHeroe` delega el efecto en `alAlcanzarAlHeroe`. Los métodos que fijan la secuencia son `final`, de modo que cada subtipo implementa únicamente sus decisiones de movimiento y encuentro.

En el Hito 1 se implementó `Errante`. En cada turno se obtiene la lista de vecinas transitables y se elige una al azar. Si la lista está vacía, se conserva la posición actual. Al alcanzar al héroe, el errante le descuenta 30 unidades de energía y vuelve a su posición inicial. El generador de números pseudoaleatorios se recibe por constructor, lo que permite reproducir el recorrido con una semilla fija.

1.6.2. Apertura de cofres y puntaje

La clase `Cofre` define la secuencia común de apertura. En el primer encuentro con el héroe se registra la apertura, se incrementa el contador de cofres abiertos y se ejecuta el efecto específico mediante `abrir`. Luego, `debeSalirDelPlano()` indica a la partida que corresponde retirar el cofre.

Cada cofre abierto aporta 100 puntos al calcular el puntaje de la partida. En `CofreDePuntos`, el método `abrir` queda vacío porque su beneficio ya está cubierto por el contador común. Así se conserva la regla de puntaje para todos los tipos de cofre.

1.6.3. Creación de entidades mediante catálogos

El JSON del plano indica dónde debe colocarse un monstruo o un cofre mediante los símbolos `M` y `C`. Para elegir el tipo concreto se implementaron `CatalogoDeMonstruos` y `CatalogoDeCofres`. Cada catálogo guarda una fábrica por nombre de tipo. Al cargar una celda, se sortea una fábrica y se le entrega la posición en la que debe crear la entidad.

Las fábricas reciben la posición como argumento y pueden conservar otros parámetros, como el daño o la instancia de `Random`. Para incorporar un tipo se implementa su comportamiento y se registra su fábrica en `CreadorPartida`. El cargador sigue utilizando la misma operación `crearAlAzar`.

A continuación, la Figura 1 muestra la jerarquía de monstruos y cofres y cómo el cargador utiliza los catálogos para crear las entidades del plano.

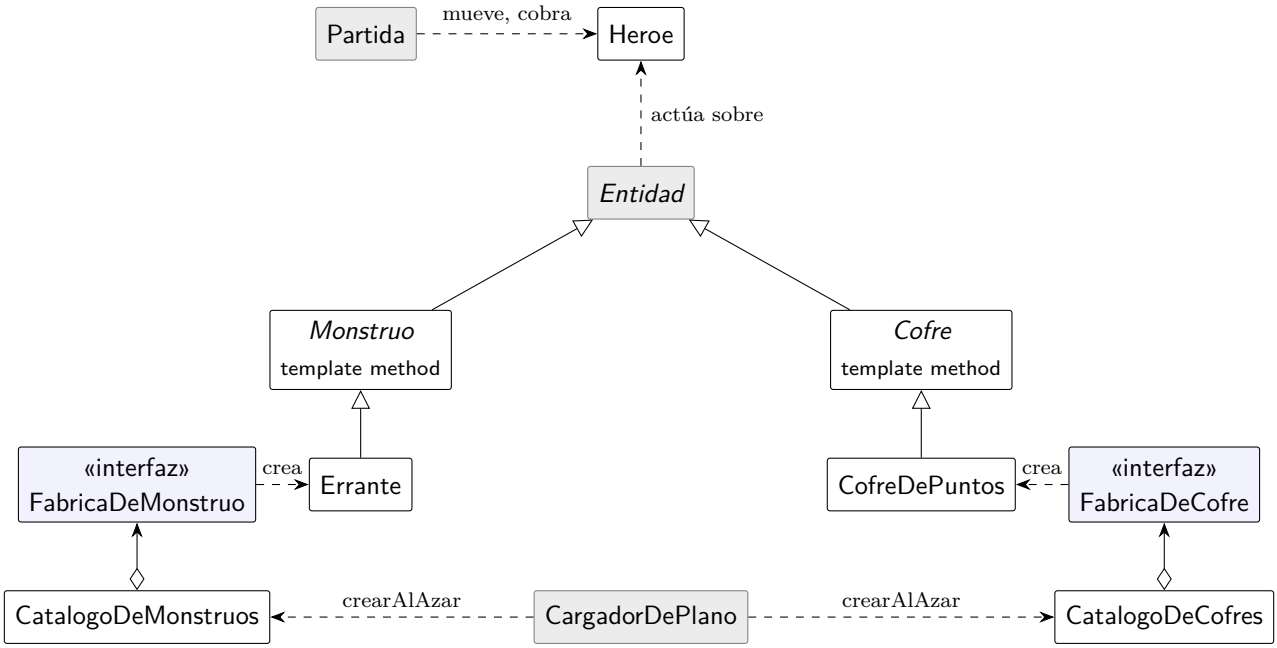


Figura 1: Entidades, fábricas y catálogos del Hito 1.

1.7. Persistencia, configuración y vista

1.7.1. Lectura de configuración

Se utilizó Gson para cargar `configuracion.json` al iniciar la partida. El archivo define la energía inicial, los costos de piso, agua y lava, una semilla opcional y un bloque por dificultad con la cantidad de tableros, pisos, filas, columnas, cofres y monstruos. Una semilla fija permite reproducir el mapa con los mismos parámetros.

1.7.2. Consola y representación del plano

Se encapsuló el acceso al teclado en `EntradaConsola`. Para elegir la dificultad se lee una línea, se eliminan los espacios externos y se comprueba que el valor sea un entero dentro del intervalo del menú. Ante una entrada incorrecta se informa el problema y se vuelve a pedir el valor.

La interfaz `Vista` permite que el controlador solicite acciones y muestre el resultado de cada paso. La implementación BMP dibuja el terreno, la salida, las entidades y el héroe en capas sobre una imagen de celdas de 16×16 píxeles. La barra de energía ocupa la fila inferior. Se escribe una imagen inicial y una por paso válido en la carpeta `salida`.

1.8. Estructuras de datos

1.8.1. Pila de excavación del generador

Se utilizó `ArrayDeque` como pila explícita para guardar el camino activo. Al excavar una celda se la apila. Cuando se agotan los destinos disponibles desde esa celda, se la desapila y se continúa desde la anterior. Las operaciones en el extremo de la pila tienen costo amortizado $O(1)$. El espacio

utilizado depende de la longitud máxima del camino activo.

1.8.2. Estructuras utilizadas por el validador

El plano se representa mediante una matriz `char[][]`, que permite consultar el símbolo de una celda a partir de su fila y columna en tiempo constante.

Para el recorrido BFS se utiliza una cola implementada con `ArrayDeque`. La cola conserva las celdas pendientes de visitar y permite procesarlas por distancia creciente desde la entrada.

Las distancias se almacenan en una matriz `int[][]`. Cada posición contiene la distancia mínima desde la entrada o -1 si no fue alcanzada.

Para un plano de f filas y c columnas, el recorrido tiene complejidad temporal $O(f \cdot c)$: cada celda alcanzada se encola una sola vez y se examinan sus cuatro vecinos. El espacio auxiliar también es $O(f \cdot c)$, considerando la matriz de distancias y la cola.

1.8.3. Estructuras del modelo y de la integración

- `Tablero` utiliza una matriz `Terreno[][]`. Consultar o modificar el terreno de una posición cuesta $O(1)$. El método `vecinasTransitables` construye una lista de hasta cuatro posiciones, por lo que también cuesta $O(1)$.
- `Plano` almacena sus entidades en una `LinkedList`. Si el plano contiene e entidades, recorrerlas o buscar las que ocupan una celda cuesta $O(e)$. Quitar una entidad determinada también cuesta $O(e)$ en el peor caso, porque primero hay que localizarla.
- `Plano.getEntidades()` devuelve una copia de la lista con referencias a las mismas entidades. Esto permite recorrerla mientras se retiran entidades del plano. Construir la copia requiere $O(e)$ de tiempo y espacio.
- `CreadorPartida` utiliza un `EnumMap` para asociar los terrenos transitables con sus costos de energía. La consulta por terreno cuesta $O(1)$.
- En las pruebas del controlador, una `ArrayDeque` almacena las acciones preparadas en el orden en que deben ejecutarse. Retirar la primera acción cuesta $O(1)$.

1.8.4. Estructuras de los catálogos y del errante

Cada catálogo utiliza un `LinkedHashMap` para asociar el nombre de un tipo con su fábrica. La estructura conserva el orden de registro. Esto permite que dos catálogos con los mismos tipos registrados en el mismo orden produzcan la misma secuencia de elecciones si reciben la misma semilla. El registro y la consulta por nombre tienen costo promedio $O(1)$.

Para realizar el sorteo, `crearAlAzar` copia las fábricas a un `ArrayList` y elige una por índice. Con k tipos registrados, construir la lista requiere $O(k)$ de tiempo y espacio auxiliar. Esta operación se realiza durante la carga del plano.

Durante la partida, `Errante` elige su próximo movimiento entre las vecinas transitables que devuelve el tablero. Como se examinan hasta cuatro posiciones ortogonales, decidir el movimiento de un errante cuesta $O(1)$ por turno.

1.9. Pruebas, integración e intercambio

1.9.1. Pruebas del generador

Se verificó el rechazo de parámetros inválidos: campos obligatorios ausentes, tipos incorrectos, dimensiones menores al mínimo, cantidades negativas, valores fraccionarios para campos enteros y solicitudes que superan la capacidad máxima. También se comprobó el rechazo de sintaxis JSON no admitida por la lectura estricta.

Las pruebas de construcción incluyen la grilla inicial, la conservación de los bordes, la excavación, las bifurcaciones y las proporciones transitables.

Se comprobó la selección de la salida mediante BFS y la distribución de monstruos, cofres, agua y lava, verificando cantidades, distancias y conservación de las posiciones ocupadas.

Las pruebas del resultado comprueban dimensiones, símbolos, coordenadas y el caso sin salida. También verifican que los mismos parámetros produzcan el mismo JSON y que dos semillas distintas produzcan resultados diferentes en un caso elegido.

Se incorporaron pruebas de regresión para dos situaciones: una grilla mínima en la que no cualquier entrada permite ubicar un monstruo, y una generación que requiere reintentar porque la primera grilla no tiene espacio para los cofres.

1.9.2. Pruebas del validador

Las pruebas de `ValidadorPlanoTest` incluyen la aceptación del ejemplo del Anexo B y casos de rechazo para cada una de las once reglas.

Cada caso de rechazo comprueba que el mensaje de la excepción identifique la regla esperada. Para ello, el plano utilizado debe satisfacer las comprobaciones que se ejecutan antes de esa regla.

También se incluyen casos de JSON mal formado, contenido que no representa un objeto y ausencia del campo `plano`.

1.9.3. Integración del generador con el validador

Se ejecutaron sesenta combinaciones de generación: veinte semillas por cada dificultad. Cada JSON obtenido se pasó al validador del proyecto y todos fueron aceptados.

Esta prueba comprueba la compatibilidad entre ambos componentes para los parámetros seleccionados. No implica que se hayan cubierto todas las dimensiones y cantidades de entidades posibles.

1.9.4. Pruebas del modelo y de la integración de la partida

Se desarrollaron 31 pruebas distribuidas en cinco clases. La Tabla 2 resume su alcance.

Clase	Pruebas	Qué verifica
`TableroTest`	10	Inicialización, terrenos, límites y vecinas transitables.
`PlanoTest`	8	Entrada, salida y operaciones sobre las entidades del plano.
`PosicionTest`	4	Desplazamientos, igualdad por valor y uso en un conjunto.
`CreadorPartidaTest`	5	Creación con el generador real, estado inicial del héroe, cantidades de entidades y uso de la semilla.
`ControladorJuegoTest`	4	Victoria, salida voluntaria, choque contra una pared y espera.

Tabla 2: Pruebas de la base del modelo y de la integración.

Para comprobar la reproducibilidad se crean dos partidas con los mismos parámetros y la misma semilla. Se comparan sus terrenos, la entrada, la salida y las entidades con sus posiciones.

El controlador se prueba con una implementación de `Vista` que entrega acciones preparadas y registra las llamadas a `mostrar` y `avisar`. Por ejemplo, al intentar mover al héroe contra una pared se comprueba que se emita un aviso, se conserve el número de paso y no se vuelva a mostrar la partida.

La Figura 2 muestra el resultado de la ejecución de las pruebas con Gradle.

```
PartidaTest > elPuntajeDeUnaPartidaSinPasosNiCofresEsLaEnergiaPorDos() PASSED

PartidaTest > pisarLaSalidaConLaUltimaEnergiaGanaEnVezDePerder() PASSED

PartidaTest > pisarLaSalidaGanaLaPartida() PASSED

PartidaTest > moverAPisoCuestaUnoYSumaUnPaso() PASSED

PartidaTest > unaPartidaTerminadaNoAceptaEsperar() PASSED

PartidaTest > siElHeroePisaAlMonstruoLoAlcanzaEnEseMismoPaso() PASSED

PartidaTest > elPuntajeUsaLaEnergiaRealDelHeroeYLosPasosDados() PASSED

PartidaTest > unaPartidaTerminadaNoAceptaMasMovimientos() PASSED

PosicionTest > moverDevuelveLaVecinaEnCadaDireccion() PASSED

PosicionTest > unHashSetNoGuardaDosVecesLaMismaPosicion() PASSED

PosicionTest > moverNoCambiaLaPosicionOriginal() PASSED

PosicionTest > dosPosicionesConLaMismaFilaYColumnaSonIgualesYTienenElMismoHash() PASSED

LectorConfiguracionTest > leeTodosLosCamposDeUnArchivoValido() PASSED

LectorConfiguracionTest > rechazaSiFaltaElCostoDeUnTerreno() PASSED

LectorConfiguracionTest > rechazaUnArchivoVacio() PASSED

LectorConfiguracionTest > rechazaEnergiaInicialCero() PASSED

LectorConfiguracionTest > rechazaUnArchivoQueNoExiste() PASSED

LectorConfiguracionTest > rechazaUnJsonMalEscrito() PASSED

LectorConfiguracionTest > leeUnaSemillaFija() PASSED

LectorConfiguracionTest > rechazaSiFaltaUnaDificultad() PASSED

VistaBmpTest > elHeroeSeDibujaEnSuCelda() PASSED

VistaBmpTest > elBmpTieneElTamanioDelPlanoMasLaBarraDeEnergia() PASSED

VistaBmpTest > despuesDeMostrarExisteElArchivoDelPaso() PASSED

VistaBmpTest > alCrearseBorraLosPasosDeUnaPartidaAnteriorYNadaMas() PASSED

VistaBmpTest > noAceptaEnergiaMaximaCero() PASSED

BUILD SUCCESSFUL in 3s
3 actionable tasks: 1 executed, 2 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.0/userguide/configuration_cache_enabling.html
PS C:\Users\Usuario\OneDrive\Escritorio\SCRIPTS\Cb100\JAVALOVERS>
```

Figura 2: Resultado de la ejecución de las pruebas con Gradle.

1.9.5. Pruebas del héroe, las entidades y los catálogos

Al cierre del Hito 1 se registraron 70 pruebas unitarias de esta parte, distribuidas según la Tabla 3. El resultado quedó documentado en el commit `c9be1c5`.

Clase	Casos	Aspectos verificados
`HeroeTest`	16	Creación, movimiento, argumentos inválidos y energía mínima igual a cero.
`ErranteTest`	17	Movimiento transitable, encierro, reproducibilidad, daño y regreso al origen.
`CofreDePuntosTest`	7	Apertura, contador de cofres y conservación de la energía y la ubicación.
`CatalogoDeMonstruosTest`	15	Registro, duplicados, catálogo vacío, posición de creación y sorteo reproducible.
`CatalogoDeCofresTest`	15	Registro, validaciones y selección entre distintos tipos de cofre.
Total	**70**	

Tabla 3: Pruebas unitarias del héroe, las entidades y los catálogos al cierre del Hito 1.

Para verificar el determinismo del movimiento se compararon dos errantes durante 50 pasos con la misma semilla. También se probó el contrato de `Monstruo` mediante un subtipo de prueba que devolvía una posición nula. En ese caso se comprobó que la clase base informara el error. Para probar la selección entre fábricas se utilizaron entidades distinguibles por su daño o por una etiqueta de prueba.

1.9.6. Pruebas de partida, configuración y vista

`PartidaTest` reúne 26 casos de movimiento, costos, espera, encuentros, estados y puntaje. `MazmorraTest` agrega ocho casos sobre la organización de planos y sus ubicaciones. Se comprueba, entre otros casos, que el tablero 2 y piso 3 corresponda al sexto plano cuando cada tablero tiene tres pisos, y que se rechace el piso cero.

En la versión integrada se identificaron ocho pruebas en `LectorConfiguracionTest` y cinco en `VistaBmpTest`. Las primeras cubren la carga y el rechazo de archivos inválidos. Las de la vista verifican la escritura del BMP, sus dimensiones con la barra de energía, la ubicación del héroe y la limpieza de imágenes de pasos anteriores.

1.9.7. Intercambio con otro grupo

Se generaron tres JSON de ejemplo, uno por dificultad, con semilla 1. Queda pendiente registrar el resultado de la validación y carga de los planos recibidos del otro grupo.

1.10. Problemas encontrados

1.10.1. Generación de planos

Una entrada podía impedir la colocación de monstruos. En las primeras pruebas, el generador elegía un piso libre como entrada y luego intentaba colocar los monstruos. En un plano pequeño, la entrada elegida podía dejar muy pocas celdas a distancia Manhattan de al menos tres, aunque otra entrada de la misma grilla permitiera ubicar todos los monstruos.

Se modificó `elegirEntrada` para evaluar los pisos candidatos antes de aceptar una posición. Cuando el plano requiere salida, también se reserva la celda que ocuparía. La selección de entrada y la colocación de monstruos utilizan la misma condición de distancia, definida en `admiteMonstruo`. Una prueba de regresión verifica el caso de una grilla mínima en la que la primera entrada evaluada resulta inadecuada.

La primera grilla podía tener pocos pisos libres. Una solicitud podía pasar la comprobación inicial de capacidad y producir una grilla con espacio insuficiente para los cofres. Esto ocurría porque la capacidad inicial expresa un máximo teórico, mientras que la excavación determina cuántos pisos tiene cada grilla concreta.

Se incorporó al ciclo de generación una comprobación que cuenta los pisos necesarios para la entrada, la salida cuando corresponde, los cofres y los monstruos. Si la grilla tiene menos pisos, se descarta y se inicia otro intento con la misma instancia de `Random`. Una prueba con semilla fija reproduce el caso en que la primera grilla resulta insuficiente y un intento posterior permite completar el plano.

1.10.2. Validación y carga de planos

Una prueba podía fallar por una regla distinta de la esperada. Al modificar el plano de ejemplo para probar una regla, el cambio podía infringir también una comprobación anterior. Por ejemplo, al cerrar celdas para reducir la proporción transitable, era posible desconectar el plano. En ese caso, el validador se detenía por falta de conectividad (R5) antes de evaluar la proporción (R7).

Se prepararon planos pequeños que satisfacen las comprobaciones anteriores a la regla bajo prueba. Además de verificar que se lance una excepción, cada prueba comprueba el identificador de la regla en el mensaje. Así se detecta si el rechazo ocurrió por el motivo esperado.

Conversión entre coordenadas del JSON e índices de la grilla. El JSON expresa la fila y la columna desde uno. El validador convierte esas coordenadas a índices desde cero para recorrer sus arreglos. Durante la carga había que distinguir entre las coordenadas declaradas y las posiciones obtenidas al recorrer las cadenas del JSON.

El cargador conserva los valores de `entrada` y `salida`, que ya utilizan la convención del modelo. Al recorrer las cadenas, suma uno a los índices de fila y columna antes de construir la `Posicion` de cada terreno o entidad.

Lectura de un plano sin salida. El formato de intercambio representa la ausencia de salida con el campo `salida` en `null`. El validador también admitía que la clave estuviera ausente cuando `conSalida`

era falso. Gson representa el primer caso con `JsonNull` y devuelve `null` de Java en el segundo. Al invocar `isJsonNull()` sobre una referencia nula, el cargador producía una `NullPointerException`.

Se ajustó la lectura para comprobar primero si la referencia es nula y luego si el elemento representa un nulo de JSON. En ambos casos, el plano se construye sin posición de salida.

1.10.3. Modelo e integración de la partida

Numeración de tableros y pisos. En una primera versión, las filas y columnas del modelo comenzaban en uno, pero `Mazmorra` numeraba los tableros y pisos desde cero. Una misma `Ubicacion` combinaba así dos convenciones, lo que dificultaba identificar el plano al que pertenecía una celda. Se unificó la numeración pública de tableros, pisos, filas y columnas desde uno.

Un cofre podía otorgar puntos más de una vez. Al volver a una celda con un cofre ya abierto, el héroe podía recibir nuevamente los puntos. Se incorporó el estado de apertura en `Cofre` y la consulta `debeSalirDelPlano()` en `Entidad`. Después de resolver el encuentro, `Partida` consulta a la entidad y retira el cofre abierto.

Cambio en el contrato de Entidad. Al agregar el método abstracto `getSimbolo()` a `Entidad`, la entidad auxiliar utilizada en `PlanoTest` dejó de compilar. Se actualizó esa clase para que implementara el nuevo método. El caso mostró que un cambio en el contrato de la clase base también alcanza a las implementaciones utilizadas por los tests.

Ejecución desde Gradle y entrada por teclado. Al iniciar el juego mediante la tarea `run`, la aplicación necesitaba recibir la entrada del teclado para leer la dificultad y las acciones. Además, Gradle ejecutaba el programa desde `app/`, mientras que `configuracion.json` se encontraba en la raíz del proyecto.

En `app/build.gradle` se configuró la tarea para utilizar `System.in` como entrada estándar y la raíz del proyecto como directorio de trabajo. Para las terminales de PowerShell que mostraban incorrectamente los acentos, se documentó en el README cómo configurar la salida en UTF-8.

Falta de respuesta ante un movimiento bloqueado. Cuando el héroe intentaba atravesar una pared, la partida rechazaba el movimiento y conservaba el estado anterior. Como el jugador no recibía ningún mensaje, era difícil distinguir ese rechazo de una tecla que no se hubiera leído.

Se agregó el método `avisar` a la interfaz `Vista`. El controlador lo invoca cuando la acción es inválida. El jugador recibe una explicación y el número de paso se conserva, por lo que tampoco se genera un BMP adicional.

1.10.4. Implementación del héroe, las entidades y los catálogos

Descuento de energía mayor que la disponible. En la primera versión, `restarEnergia` lanzaba una excepción cuando el descuento superaba la energía del héroe. Esto interrumpía una situación prevista por las reglas, como recibir un ataque de 30 unidades con solo 20 disponibles. Se corrigió el método para dejar la energía en cero y permitir que la partida resuelva el resultado del turno. Se mantuvo el rechazo de descuentos negativos.

Subtipos incompletos y compilación de pruebas. Durante el desarrollo se crearon clases de hitos posteriores que extendían `Monstruo` o `Cofre` sin implementar todavía sus métodos abstractos. Como Gradle compila el código principal antes de ejecutar las pruebas, estas clases impedían probar incluso componentes ya terminados. Se postergó la incorporación de esas clases a la jerarquía hasta contar con sus implementaciones. También se corrigió una firma duplicada que difería únicamente en una mayúscula.

Selección de una fábrica por índice. El método `values()` del mapa devuelve una colección de fábricas sin acceso por índice. Para sortear una fábrica se construyó una lista con los valores en orden de registro y se seleccionó una posición mediante el `Random` recibido. El catálogo quedó encargado de registrar y elegir las fábricas, y cada fábrica quedó encargada de construir una entidad.

1.11. Cuestionario

Pregunta 5. Profundidad de la excavación y conectividad. En un tablero de 25×40, la excavación comienza en $(1, 1)$ usando índices desde cero. Al avanzar de a dos casillas dentro del borde se obtienen 12 filas y 19 columnas candidatas, es decir, 228 destinos posibles contando el inicial. La profundidad depende de la semilla y queda acotada por 228 posiciones activas. Una versión recursiva ocuparía un marco de la pila de la JVM por nivel. En tableros mayores podría agotarse esa pila; la versión iterativa guarda el camino en un `ArrayDeque`.

La excavación abre la pared intermedia entre cada destino nuevo y una celda ya alcanzada. Así se conserva la conectividad desde el primer tramo. La cantidad de bifurcaciones obtenida depende del recorrido. Si resulta inferior al mínimo exigido, se abren paredes interiores que comunican caminos existentes.

Pregunta 6. Tamaño de la película en BMP y PNG. Con celdas de 16×16 píxeles, la grilla de 25×40 ocupa 640×400 píxeles. En un BMP de 24 bits, cada fila ocupa 640×3 bytes y ya está alineada a cuatro bytes. Con una cabecera de 54 bytes, el archivo ocupa 768 054 bytes. La vista del juego agrega una fila para la barra de energía. El BMP producido mide 640×416 y ocupa 798 774 bytes.

En una partida de 300 pasos se generan 301 imágenes contando el estado inicial. En el formato del juego, la película ocupa 240 430 974 bytes. Al convertir a PNG la imagen inicial de la prueba con semilla 42 se obtuvieron 2 837 bytes. El tamaño de los demás PNG depende del contenido de cada paso. Las áreas de color uniforme favorecen la compresión sin pérdida de PNG. El BMP utilizado almacena los píxeles directamente, de manera análoga a las muestras de audio en un WAV con PCM. MP3 también reduce el tamaño, pero utiliza compresión con pérdida.

Pregunta 8. Incorporación de una ventana gráfica. Para incorporar una ventana gráfica se implementa la interfaz `Vista`. El método `mostrar` dibuja el estado de traduce la entrada del jugador a una `Accion` y `avisar` presenta los mensajes. La implementación se selecciona en `Main`. El controlador ya delega estas tres operaciones en la vista, y las reglas de siguen a cargo de `Partida`.

1.12. Limitaciones

1.12.1. Generación de planos

La generación utiliza una búsqueda aleatoria acotada a cien intentos. Alcanzar ese límite indica que no se encontró una solución durante la ejecución, pero no demuestra que la solicitud sea imposible.

La validación inicial de capacidad permite rechazar solicitudes que exceden el espacio máximo disponible. Sin embargo, esa comprobación no garantiza por sí sola que exista una distribución que satisfaga todas las restricciones geométricas.

Las pruebas incluyen casos límite y sesenta combinaciones aceptadas por el validador. Esta cobertura no constituye una verificación exhaustiva de todas las entradas posibles.

1.12.2. Alcance de la integración en el Hito 1

En el Hito 1 se completó el recorrido jugable de la dificultad Fácil. El creador puede construir los tres planos de Medio y los nueve de Difícil. Para recorrerlos y alcanzar la salida quedan por incorporar las escaleras, los pasillos y las pruebas de movimiento entre planos.

El controlador reconoce las acciones de usar objetos y guardar. Su ejecución corresponde a las etapas de inventario y persistencia. La clase `Agenda` se incorporó a las firmas de las entidades como base para los efectos temporales.

1.12.3. Alcance de las entidades en el Hito 1

En el Hito 1 se registraron el errante y el cofre de puntos. El daño del errante se entrega por constructor al registrar su fábrica. Su movimiento considera la transitabilidad del terreno, por lo que puede compartir una celda con otra entidad. Los encuentros reciben una `Agenda`, prevista para incorporar los efectos temporales en los hitos posteriores.

1.13. Aportes individuales

1.13.1. Presas, Juan Cruz

Se diseñó e implementó la clase `ValidadorPlano`, incluida la lectura del JSON y la verificación de las once reglas del Anexo B. Para comprobar la conectividad del plano (R5) y la distancia máxima de la salida (R9), se utilizó un único recorrido BFS desde la entrada. Ante una infracción, se informa la regla incumplida en el mensaje de la excepción.

Se desarrollaron las pruebas del validador, con un caso de aceptación para el plano de ejemplo del anexo, un caso de rechazo por cada regla y casos de JSON inválido. En cada prueba por regla se verificó que el rechazo correspondiera a la infracción esperada.

En conjunto con Facundo Solari Parravicini, se diseñó e implementó `CargadorDePlano`. La clase valida el JSON, construye el tablero, registra la entrada y la salida, y crea los cofres y monstruos mediante los catálogos del juego. Durante la integración se corrigió la lectura de planos sin la clave `salida`.

1.13.2. Martinez, Demian Arian

Se implementó `Heroe` con su ubicación, energía y contador de cofres abiertos. Se desarrollaron las jerarquías de monstruos y cofres sobre la clase `Entidad`, junto con los tipos `Errante` y `CofreDePuntos` requeridos para el Hito 1.

Se implementaron las interfaces de fábrica y los catálogos de monstruos y cofres. Se definieron el registro de tipos, la validación de nombres y la selección reproducible de entidades para que el cargador pueda crearlas en las posiciones indicadas por el plano.

Se desarrollaron 70 pruebas unitarias del héroe, el errante, el cofre de puntos y ambos catálogos. Entre los commits representativos se encuentran `50b43dd` para el errante, `faf39f6` para los cofres, `4133bd4` para el catálogo de monstruos y `c9be1c5` para el cierre del hito. Se redactó la documentación de diseño y extensión de estas clases.

1.13.3. Solari Parravicini, Facundo

Se diseñó e implementó `GeneradorGrupoNN` según el Anexo B. Se incorporó la lectura estricta del JSON y se generaron los caminos mediante backtracking con una pila explícita. A partir de la semilla recibida, se ajustaron las bifurcaciones y la proporción transitable, se distribuyeron las entidades y los terrenos, y se generó el JSON de salida.

Se seleccionó una entrada compatible con la distancia Manhattan exigida para los monstruos y, cuando se solicitó una salida, se utilizó BFS para ubicarla a distancia máxima de la entrada. Se incorporaron reintentos para las grillas con espacio insuficiente o sin una entrada adecuada.

Se desarrollaron pruebas de determinismo, parámetros inválidos, distribución de entidades y terrenos, y casos límite. Se integró el generador con `ValidadorPlano` mediante veinte semillas por dificultad. Los sesenta planos generados fueron aceptados.

En conjunto con Juan Cruz Presas, se diseñó e implementó `CargadorDePlano` para convertir el JSON validado en un `Plano` del juego.

Se recopilaron los aportes de los integrantes y se integraron en el informe y los manuales del TP. Se organizó el contenido en LaTeX y se unificaron el formato y la redacción del documento.

1.13.4. Caldas Canedo, Owen

Se desarrollaron la representación inicial de la configuración y su lector JSON con Gson, la interfaz de vista, el exportador BMP y la entrada por consola. Se encapsuló el acceso al teclado mediante un único lector de líneas y se incorporaron reintentos para valores que no son enteros o quedan fuera del intervalo del menú.

Los commits `4338d7e`, `f82b1b4` y `d5e17b6` corresponden a configuración, vista y consola, respectivamente. A partir de esta base, durante la integración se adaptaron el lector y el dibujo en capas al modelo de la partida.

1.13.5. Burgos, Matías Alejandro

Se diseñaron e implementaron `Partida` y `Mazmorra`, junto con `Accion`, `EstadoPartida` y `ResultadoTurno`. Se resolvieron el movimiento, el costo por terreno, la espera, el turno de los monstruos, las condiciones

de victoria y derrota y el cálculo del puntaje.

Se corrigió la numeración de tableros y pisos para que comenzara en uno y se adaptaron las pruebas que construían ubicaciones. Se desarrollaron 34 pruebas unitarias, 26 de partida y ocho de mazmorra. Para probar el movimiento durante el desarrollo se utilizó una primera versión mínima del héroe, reemplazada al integrarse la implementación de Demian.

Los cambios se integraron mediante los PR #5 y #14. Entre los commits representativos se encuentran `434311f` (puntaje), `8cd33af` (mazmorra), `192508e` (movimiento y costos), `a2735a5` (espera y turno de monstruos) y `799cc3c` (numeración desde uno). También se desarrolló un prototipo de controlador con entrada por consola en `a8de4f7`. Para la entrega se integró el controlador de Dan.

1.13.6. Ortega, Dante Luca

Se implementó la base del modelo: los valores `Direccion`, `Posicion`, `Ubicacion` y `Dificultad`, las clases `Tablero`, `Plano` y `Entidad`, el tipo `Terreno` y la estructura inicial de `Agenda`.

Para el Hito 1 se integró la creación y ejecución de la partida mediante `CreadorPartida`, `ControladorJuego` y `Main`. También se configuró la ejecución con Gradle y se desarrollaron 31 pruebas para el modelo, el creador y el controlador.

Entre los commits representativos se encuentran `fcb23ea` para `Tablero`, `4d88bd0` para `Plano`, `b98f765` para el símbolo de las entidades, `0de4f8a` para `CreadorPartida`, `772a562` para el controlador y `37c150d` para la configuración de Gradle.

Como mejoras para futuros trabajos, se propone acordar y documentar la convención de coordenadas desde el comienzo y dividir los cambios que dependen de otros integrantes en PR más pequeños. Al extender `Entidad` con `getSimbolo()`, se tuvo que actualizar la entidad auxiliar de `PlanoTest`, además de las implementaciones utilizadas en el juego. Recibir la vista y el generador por constructor facilitó las pruebas de integración.

1.14. Conclusiones

1.14.1. Resultados del Hito 1

Se completó una partida jugable en dificultad Fácil, desde la lectura de la configuración hasta la victoria o la derrota. El contrato JSON permitió integrar el generador con el validador y el cargador. Se probaron veinte semillas por dificultad y los sesenta planos fueron aceptados por el validador.

Con la semilla 42 se verificaron ambos resultados desde la consola. En una partida se alcanzó la salida en 87 pasos, con cuatro cofres abiertos y siete unidades de energía. En otra se agotó la energía en el paso 100. Al contar la imagen inicial, se generaron 88 y 101 archivos BMP, respectivamente.

2. Manual de usuario

2.1. Descripción del programa

El objetivo es guiar al héroe hasta la salida de la mazmorra. Cada movimiento consume energía según el terreno y los monstruos pueden reducirla al alcanzar al héroe. Cada cofre abierto aporta 100 puntos al puntaje final. En la dificultad Fácil, la entrada y la salida se encuentran en el mismo plano.

Las acciones se ingresan por consola. Para consultar el mapa, la ubicación del héroe y la barra de energía, se abre la imagen BMP del último paso.

2.2. Requisitos, instalación y ejecución

Desde la raíz del proyecto, abrir PowerShell y ejecutar:

```
.\gradlew.bat run --console=plain
```

Al aparecer el menú, ingresar el número de la dificultad y presionar Enter. Para jugar la versión del Hito 1, elegir Fácil.

2.3. Inicio de partida y selección de dificultad

Al iniciar el programa, ingresar el número correspondiente a Fácil y presionar Enter. La consola muestra el estado inicial y el juego genera el archivo `salida/paso-0000.bmp`. Abrir esa imagen para localizar al héroe y elegir el primer movimiento.

2.4. Pantallas, controles y reglas básicas

Ingresar una tecla y presionar Enter para ejecutar la acción.

Tecla	Acción
`w` / `s`	Mover hacia arriba / abajo.
`a` / `d`	Mover hacia la izquierda / derecha.
`e`	Esperar un turno. Los monstruos pueden moverse.
`q`	Salir de la partida.

En el mapa, el héroe se muestra en rojo, la salida en verde, los cofres en amarillo y los monstruos en magenta. Las paredes son gris oscuro y el piso es gris claro. El agua se representa en azul y la lava en naranja. La barra debajo del tablero indica la energía restante.

2.5. Tareas frecuentes

Consultar el último paso. Abrir la carpeta `salida` y elegir el BMP con el número más alto. Cada movimiento válido o espera genera la imagen siguiente.

Abrir un cofre. Mover al héroe hasta una celda amarilla. Al ingresar, el cofre se abre, desaparece del plano y se contabiliza para el puntaje final.

Repetir un mapa. Conservar la dificultad, la semilla y los demás parámetros de generación en `configuracion.json`. Para guardar las imágenes de la partida anterior, copiarlas a otra carpeta antes de iniciar una nueva.

2.6. Archivos y configuración

La configuración se encuentra en `configuracion.json`, en la raíz del proyecto. Los cambios se aplican al iniciar una partida. El campo `energiaInicial` indica la energía de partida y `costos` fija el consumo al ingresar a piso, agua o lava. Los valores del Hito 1 son 1, 3 y 8, respectivamente.

En `dificultades`, cada nivel indica la cantidad de tableros, pisos, filas, columnas, cofres y monstruos. Para repetir un mapa se conservan la dificultad, estos valores y una semilla fija. Con `semilla` en `null` se sortea una semilla al comenzar.

2.7. Mensajes de error y soluciones

Movimiento bloqueado. Si el destino es una pared, la consola informa que la acción no se pudo realizar. El héroe conserva su posición y el número de paso. Consultar el último BMP para elegir otra dirección.

Configuración no encontrada. Verificar que `configuracion.json` se encuentre en la raíz del proyecto y ejecutar el comando de inicio desde esa carpeta.

Acciones reservadas. En el Hito 1, las teclas `g` y `u` están reservadas para guardar y usar objetos. Al ingresarlas, se conserva el estado de la partida.

3. Manual del programador

3.1. Preparación del entorno

3.1.1. Compilación y ejecución con Gradle

El proyecto utiliza JDK 25 y el Gradle Wrapper. Desde la raíz del repositorio, en PowerShell, se pueden ejecutar los siguientes comandos:

```
.\gradlew.bat build
.\gradlew.bat test
.\gradlew.bat run --console=plain
```

En Linux y macOS se utilizan las mismas tareas con `./gradlew`. La clase principal es `ar.uba.fi.cb100.tp2.aplicacion.Main`.

La tarea `run` de `app/build.gradle` utiliza `System.in` para recibir el teclado y la raíz del proyecto como directorio de trabajo. Desde allí se lee `configuracion.json` y se generan los BMP en la carpeta `salida`. La opción `-console=plain` facilita la lectura de las preguntas del juego en la terminal.

Si PowerShell muestra incorrectamente los acentos, se puede configurar la codificación antes de iniciar el juego:

```
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
```

3.2. Estructura del proyecto

El código del juego se encuentra bajo `app/src/main/java/ar/uba/fi/cb100/tp2`. Las pruebas siguen la misma organización de paquetes bajo `app/src/test/java`.

`aplicacion` Punto de entrada, creación de la partida y controlador del ciclo de juego.

`generador` Interfaz de intercambio y generación del JSON de un plano.

`construccion` Validación del JSON y construcción del plano del modelo.

`modelo` Coordenadas, tablero, planos, héroe, entidades, catálogos y reglas de la partida.

`persistencia` Lectura de la configuración de inicio.

`vista` Entrada por consola y representación de la partida mediante imágenes BMP.

3.3. Arquitectura y recorrido de una acción

3.3.1. Recorrido de un movimiento

Al ingresar `d` y presionar Enter, se produce la siguiente secuencia:

1. La vista devuelve `Accion.MOVER_DERECHA`.
2. El controlador ejecuta `partida.moverHeroe(Direccion.DERECHA)`.
3. La partida calcula el destino y consulta al tablero si es transitable. Si está bloqueado, devuelve un resultado inválido y el controlador llama a `vista.avisar`.
4. Si el movimiento es válido, se aplica el costo del terreno, se desplaza al héroe y se incrementa el número de paso. La partida resuelve los encuentros en la celda y retira las entidades que indican que deben salir del plano.
5. Si la partida continúa en curso, se procesa el turno de las entidades del plano actual.
6. El controlador llama a `vista.mostrar` para presentar el estado resultante. Si se produjo una victoria o una derrota, finaliza el ciclo y `Main` informa el resultado.

3.4. Modelo y ejecución de la partida

3.4.1. Uso de posiciones y entidades

Al consultar o modificar el terreno se pasa al tablero una `Posicion` con fila y columna desde uno. El propio `Tablero` realiza la conversión a índices del arreglo. Para preparar un tablero de prueba deben abrirse las celdas necesarias mediante `setTerreno`, ya que todas comienzan en pared.

Para incorporar o retirar una entidad se usan `Plano.agregarEntidad` y `Plano.quitarEntidad`. El método `getEntidades()` devuelve una copia de la lista. Las entidades de la copia son los mismos objetos que están en el plano.

3.4.2. Contrato de Entidad

- `actuar(plano, heroe)` define el comportamiento de la entidad durante su turno.
- `alEncontrarseConHeroe(heroe, agenda)` resuelve el encuentro con el héroe.
- `getSimbolo()` devuelve el símbolo utilizado por la vista. Las clases `Cofre` y `Monstruo` fijan los símbolos `C` y `M`, respectivamente.
- `debeSalirDelPlano()` devuelve `false` por defecto. Los cofres utilizan su estado de apertura para indicar cuándo deben retirarse.

La posición se actualiza desde la propia entidad mediante `setPosicion`, que es protegido. Si se agrega un método abstracto a `Entidad`, también deben revisarse las implementaciones auxiliares utilizadas por los tests.

3.4.3. Acceso a planos y resultado de una acción

Para acceder al plano indicado por una ubicación se utiliza `Mazmorra.getPlano`. Para un tablero t, un piso p y n pisos por tablero, el índice en la lista es $(t-1)n+(p-1)$. La mazmorra rechaza tableros o pisos inexistentes, incluido el piso cero.

`moverHeroe` y `esperar` devuelven un `ResultadoTurno` con la validez de la acción y el estado de la partida. El controlador utiliza ese resultado para decidir si dibuja el paso o avisa al jugador. Al modificar el cierre del turno debe conservarse la prioridad de la victoria sobre la derrota al llegar a la salida.

3.5. Generador de laberintos

3.5.1. Ubicación y dependencias

El módulo se encuentra en el paquete `ar.uba.fi.cb100.tp2.generador`. La implementación es `GeneradorGrupoNN` y utiliza Gson para leer y escribir JSON.

Los records anidados `Celda` y `Parametros` representan las coordenadas internas y los parámetros de generación.

3.5.2. Contrato e invocación

El método `generar` recibe un texto JSON con el siguiente formato:

```
{
  "version": 1,
  "dificultad": "FACIL",
  "filas": 25,
  "columnas": 40,
  "semilla": 42,
  "cofres": 12,
  "monstruos": 4,
  "conSalida": true
}
```

La lectura utiliza Gson con `Strictness.STRICT`. Los valores numéricos destinados a campos enteros se convierten de forma exacta, rechazando partes fraccionarias y valores fuera del rango admitido.

Todos los campos son obligatorios. La versión admitida es 1; las dimensiones deben ser enteros mayores o iguales a cinco; las cantidades de entidades deben ser enteros no negativos. La dificultad debe ser `FACIL`, `MEDIO` o `DIFICIL`, la semilla un `long` y `conSalida` un booleano.

Las cantidades corresponden a un único plano. Si la configuración general contiene una semilla nula, el programa debe resolver su valor antes de esta llamada.

También se comprueba una cota de capacidad para las entidades, considerando el interior del plano, el máximo transitable y las posiciones reservadas.

```
    GeneradorDeLaberinto generador = new GeneradorGrupoNN();
    String planoJson = generador.generar(parametrosJson);
```

El resultado contiene `version`, `generador`, `parametros`, `entrada`, `salida` y `plano`. Este último es un arreglo de cadenas, una por fila. Los parámetros conservan los campos y valores de la entrada.

Las coordenadas de entrada y salida comienzan en uno en el JSON y en cero dentro del código. Cuando no se solicita salida, el campo correspondiente vale `null` y no se coloca el símbolo `S`.

El resultado debe pasar por el validador antes de cargarse en el modelo.

3.5.3. Modificación de los terrenos

Para cambiar la distribución de agua y lava se modifica el método `colocarAguaYLava`. Debe conservarse el máximo combinado del 25 % transitable y la selección exclusiva de pisos libres.

Después se actualizan las cantidades esperadas en el test parametrizado de terrenos. Las verificaciones de posiciones ocupadas y del límite combinado deben mantenerse.

3.5.4. Sustitución del generador

Se incorpora la implementación recibida al paquete del generador y se reemplaza la clase instanciada en el punto de creación. La interfaz y el formato de intercambio no cambian.

Antes de integrar el componente se deben validar sus archivos de ejemplo y ejecutar las pruebas de generación y carga.

3.5.5. Consideraciones para modificar el código

Los métodos que toman decisiones aleatorias reciben la misma instancia de `Random`. No se debe crear una instancia sin semilla dentro de esos métodos, ya que se perdería la reproducibilidad.

La selección de salida utiliza cantidad de movimientos, no costo energético. Reemplazar piso por agua o lava no modifica las distancias de ese BFS.

Las coordenadas deben convertirse a base uno únicamente al construir el JSON de salida.

En `Main` se selecciona el generador. El constructor de `CreadorPartida` lo recibe mediante la interfaz `GeneradorDeLaberinto`.

3.5.6. Selección de entrada y reintentos

El método `elegirEntrada` comprueba que existan suficientes posiciones para los monstruos antes de aceptar una entrada. Utiliza `admiteMonstruo` para aplicar la misma condición de distancia que `colocarMonstruos`.

Si se solicita salida, su posición se excluye de los candidatos para monstruos. Cuando ninguna entrada permite satisfacer el pedido, `elegirEntrada` devuelve `null` y `generar` inicia otro intento.

El ciclo de generación también descarta grillas que incumplen las condiciones geométricas o no tienen suficientes pisos para todas las entidades. La constante `MAX_INTENTOS` establece el límite de cien intentos.

La instancia de `Random` se crea antes del ciclo. No debe reiniciarse con la misma semilla en cada intento, porque eso repetiría las mismas decisiones.

3.5.7. Errores

El método `generar` utiliza `IllegalArgumentException` para rechazar JSON inválido, parámetros que incumplen el contrato y solicitudes que superan la cota de capacidad calculada.

Cuando no se obtiene un plano tras cien intentos, lanza `IllegalStateException`. Este error debe distinguirse de un parámetro inválido: expresa el agotamiento de la búsqueda.

Los métodos de colocación comprueban si hay pisos suficientes. Si faltan posiciones, lanzan `IllegalStateException`. Durante la generación se verifican previamente el espacio total y las posiciones disponibles para los monstruos.

3.6. Validador y cargador de planos

3.6.1. Ubicación y responsabilidades

El validador y el cargador pertenecen al paquete `ar.uba.fi.cb100.tp2.construccion`.

El validador comprueba las reglas del plano. El cargador invoca esa validación y construye los objetos del modelo utilizando los catálogos recibidos.

3.6.2. Contrato del validador

El método público recibe el JSON completo de un plano:

```
new ValidadorPlano().validar(planoJson);
```

El método devuelve `void`. Si detecta una infracción, lanza `IllegalArgumentException` con un mensaje que identifica la regla incumplida. La ejecución se interrumpe ante el primer error detectado.

Las comprobaciones son:

Regla	Criterio de validación
R1	Dimensiones de la grilla y tamaño mínimo.
R2	Uso exclusivo de los símbolos admitidos.
R3	Borde formado por paredes.
R4	Cantidad de entradas y salidas, y coincidencia con las coordenadas declaradas.
R5	Accesibilidad de todas las celdas transitables desde la entrada.
R6	Al menos un 10 % de celdas transitables con tres o más vecinos transitables.
R7	Proporción transitable entre el 30 % y el 60 % del total de celdas.
R8	Cantidades de cofres y monstruos iguales a las solicitadas.
R9	Salida a distancia mínima máxima desde la entrada, admitiendo empates.
R10	Agua y lava, en conjunto, hasta un 25 % de las celdas transitables.
R11	Monstruos a distancia Manhattan de al menos tres respecto de la entrada.

Tabla 4: Reglas de validación de los planos.

Las distancias de R9 se miden en movimientos ortogonales, sin considerar el costo energético del terreno.

3.6.3. Contrato del cargador

El cargador recibe dos catálogos y una instancia de `Random`. Los tipos disponibles deben registrarse antes de cargar un plano que contenga esas entidades.

```
CargadorDePlano cargador = new CargadorDePlano( catalogoDeCofres,
   catalogoDeMonstruos, azar);

Plano plano = cargador.cargar(planoJson);
```

La llamada a `cargar` realiza la validación, construye el tablero y agrega las entidades. No es necesario invocar al validador previamente para utilizar este método.

Si se solicita crear una entidad y su catálogo está vacío, el catálogo lanza `IllegalStateException`.

El cargador trabaja sobre un único plano. Las conexiones entre planos y la construcción del mundo corresponden a una etapa posterior.

3.6.4. Interpretación del JSON

Se utiliza el formato de salida descrito en la sección del generador y en el Anexo B.3.

El campo `plano` contiene una cadena por fila. Los campos `entrada` y `salida` deben coincidir con las marcas de la grilla. Las coordenadas comienzan en uno y una salida inexistente se representa con `null`.

Los símbolos se convierten de la siguiente manera:

- `#`: `Terreno.PARED`.
- `~`: `Terreno.AGUA`.
- `^`: `Terreno.LAVA`.
- `.`, `E`, `S`, `C` y `M`: `Terreno.PISO`.

Además, cada `C` o `M` produce una entidad mediante el método `crearAlAzar` de su catálogo. Las marcas `E` y `S` se conservan como posiciones del plano, no como tipos de terreno.

3.6.5. Modificación de las comprobaciones

Para modificar una regla se actualiza su método de validación y se conserva la identificación correspondiente en los mensajes de error.

Si una comprobación necesita distancias desde la entrada, debe reutilizar la matriz calculada por `calcularDistancias()`, evitando repetir el BFS.

Los tests deben incluir un caso aceptado y un caso rechazado que alcance la comprobación modificada. También deben ejecutarse las pruebas de integración con el generador.

Las once reglas forman parte del contrato de intercambio. Una restricción adicional propia del juego debe distinguirse de ese contrato para no rechazar planos válidos de otro grupo.

3.6.6. Consideraciones de mantenimiento

Las coordenadas del modelo comienzan en uno. Los índices internos del validador comienzan en cero y no deben trasladarse directamente al tablero.

En la matriz de distancias, las paredes conservan el valor -1. Por eso, encontrar ese valor no demuestra por sí solo que el plano sea inconexo.

Para reproducir el sorteo de tipos deben conservarse la semilla, el orden de registro de los catálogos y la secuencia de llamadas al generador de números pseudoaleatorios.

Al interpretar un campo opcional debe distinguirse `null` de Java de `JsonNull`.

3.7. Monstruos, cofres y catálogos

3.7.1. Agregar un tipo de monstruo

Se crea una subclase de `Monstruo` en el paquete `modelo.monstruos`. El constructor recibe la posición inicial y el daño, y los entrega a la clase base. Si el movimiento requiere azar, también se recibe la instancia de `Random` de la partida.

El método `decidirMovimiento(Plano, Posicion)` devuelve la próxima posición. Para permanecer en la celda actual se devuelve esa misma posición. Si se devuelve `null`, el método `actuar` lanza `IllegalStateException`. La clase base aplica el movimiento y el subtipo determina qué destinos admite.

El método `alAlcanzarAlHeroe(Heroe, Agenda)` define el efecto del encuentro. Para descontar el daño y regresar a la posición inicial se puede utilizar `atacarYVolver(heroe)`.

La fábrica se registra en `crearCatalogoDeMonstruos` de `CreadorPartida`. El registro del errante tiene la siguiente forma:

```
    catalogo.registrar("ERRANTE", p -> new Errante(p, 30, azar));
```

3.7.2. Agregar un tipo de cofre

Se crea una subclase de `Cofre` en el paquete `modelo.cofres`. Su constructor entrega la posición a la clase base y su método `abrir(Heroe, Agenda)` implementa el efecto específico. La clase base registra la apertura y permite que la partida retire el cofre. El contador se incrementa una sola vez.

La fábrica se registra en `crearCatalogoDeCofres` de `CreadorPartida`. Para un cofre cuyo constructor recibe únicamente la posición puede usarse una referencia al constructor:

```
    catalogo.registrar("PUNTOS", CofreDePuntos::new);
```

3.7.3. Contrato de los catálogos

Cada fábrica crea una entidad nueva en la posición solicitada. El método `registrar` rechaza nombres nulos, vacíos o repetidos y fábricas nulas mediante `IllegalArgumentException`. Si se invoca

`crearAlAzar` sobre un catálogo vacío, se lanza `IllegalStateException`.

Para reproducir una selección deben conservarse la semilla, el orden de registro y la secuencia de llamadas al generador de números pseudoaleatorios. Los símbolos `M` y `C` están definidos en las clases base de cada familia y se comparten entre sus subtipos.

3.7.4. Pruebas de un tipo nuevo

Para un monstruo se comprueban el movimiento en un tablero pequeño, el comportamiento cuando no tiene destinos disponibles y el efecto al alcanzar al héroe. Para un cofre se verifican su efecto, el incremento del contador y su retiro después de abrirse. En ambos casos se comprueba que la fábrica cree la entidad en la posición solicitada.

3.8. Persistencia, configuración y vistas

3.8.1. Lectura y validación

`LectorConfiguracion.leer` abre el archivo como texto UTF-8 mediante `Files.newBufferedReader` y usa Gson para construir la configuración. Luego se comprueban la energía inicial positiva, las claves de costo y la presencia de las dificultades esperadas. Si falta el archivo, falla la lectura o se detecta una configuración inválida, se informa el motivo mediante `IllegalStateException`.

3.8.2. Entrada y dibujo

`EntradaConsola` utiliza `Scanner`. La operación `leerEnteroEnRango` permite reutilizar la validación del menú. La implementación de consola comparte la entrada entre el menú y la vista.

`VistaBmp` dibuja con `Graphics2D` sobre una imagen RGB y escribe el archivo con `ImageIO.write`. El contador comienza en `paso-0000.bmp`. Para incorporar otra vista se implementan `mostrar`, `leerAccion` y `avisar`, y se selecciona la nueva instancia en `Main`.

3.9. Ejecución y organización de pruebas

3.9.1. Pruebas del generador

La clase `GeneradorGrupoNNTest` se encuentra en `app/src/test/java`, en el mismo paquete que la implementación.

Para ejecutarla desde la raíz del proyecto, en Windows:

```
.\gradlew.bat :app:test --tests
   "ar.uba.fi.cb100.tp2.generador.GeneradorGrupoNNTest"
```

Los casos utilizan semillas fijas para permitir su reproducción. Las pruebas de distribución comprueban cantidades y restricciones, sin exigir posiciones aleatorias concretas.

Al modificar la generación se deben conservar las pruebas de determinismo y agregar casos que comprueben el comportamiento afectado por el cambio.

La prueba parametrizada de integración ejecuta veinte semillas por dificultad y pasa cada JSON generado a `ValidadorPlano`.

Al modificar la selección de entrada o el ciclo de intentos, deben conservarse las pruebas de regresión para grillas mínimas y para falta de espacio en la primera grilla.

La prueba de semillas diferentes verifica un caso concreto. No debe generalizarse como una exigencia de que toda pareja de semillas produzca planos distintos.

3.9.2. Pruebas del modelo y del controlador

Para ejecutar una clase de pruebas desde PowerShell se puede utilizar el filtro `-tests`:

```
.\gradlew.bat :app:test --tests "*TableroTest"
.\gradlew.bat :app:test --tests "*ControladorJuegoTest"
```

Para agregar un caso al controlador se prepara una secuencia de acciones con la vista de prueba de `ControladorJuegoTest`. Al terminar, se comprueban el estado de la partida, las llamadas a `mostrar` y los avisos emitidos. Esto permite probar entradas como esperar, salir o intentar atravesar una pared sin depender del teclado.

3.10. Pendientes y problemas conocidos

3.10.1. Estado al cierre del Hito 1

En la versión del Hito 1, la dificultad Fácil permite completar una partida. Para Medio y Difícil queda pendiente conectar los planos y probar el recorrido entre la entrada del mundo y la salida. Las acciones de guardar y usar objetos requieren integrar la persistencia y el inventario.

Se prepararon tres JSON de ejemplo con semilla 1 para el intercambio. Una vez recibidos los planos del otro grupo, corresponde registrar el resultado de su validación y carga.